ServiGen Security Testing Report
Deep Authenticated Testing

Target: https://servi-gen.vercel.app

Backend: https://servigen-backend.onrender.com

Date: 1 November 2026

Credentials Used: All 5 roles provided (Admin, Service Agent, Approver, Service User, Support Team User)

Done By: Mageshwaran Suresh , BA Team

Summary of Bugs Found

# Severity	Bug	Status
1.High	Broken Access Control – Service Agent & Support can view all users + admin features	Confirmed
2.High	No Rate Limiting / Account Lockout on Login	Confirmed
3.Medium	JWT Token stored in localStorage	Confirmed
4.Low-Medium	Stored HTML/JS payloads accepted in Major Incidents	Confirmed
5.Low-Medium	Public Health endpoints leak information	Confirmed
6.Low	Support password note + role inconsistencies	Noted

Bug 1: Broken Access Control (High)

What is the bug?
A user with only **Service Agent** role can open the **Organisations → Users List** page and see full personal details of every user in the system (including the Admin). The same user can also open the "+ Add User" form.

This is a serious privilege escalation / excessive data exposure issue.

Logged in as **Alex Agent (Service Agent)**. You can clearly see:

- Full names, emails, phone numbers, network logins (CORP\...)
- Admin account is visible
- "+ Add User" button is present

Also confirmed via API:
Both Service Agent and Support Team User get full user data when calling GET /api/users.

How to fix it (in words):

1. On the backend, check the user's role before returning data from /api/users. Only allow the Admin role.
2. On the frontend, hide the "Organisations" menu item and the "+ Add User" button for anyone who is not an Admin.
3. Apply the same rule to other sensitive pages (CMDB, etc.) if Agents should not have access.

Bug 2: No Rate Limiting on Login (High)

What is the bug?
You can try wrong passwords as many times as you want on the login page. There is no lockout, no delay, and no CAPTCHA. This makes brute-force attacks easy.

How to fix it (in words):

- After 5 failed attempts, temporarily lock the account or add a delay.
- After a few more failures, show a CAPTCHA.
- Log failed attempts and alert the security team if too many come from the same IP.

Bug 3: JWT Stored in localStorage (Medium)

What is the bug?
After login, the application saves the authentication token (servigen_auth_token) in the browser's localStorage.

If any XSS vulnerability is ever found, an attacker can easily steal this token and take over the user's session.

How to fix it (in words):
Stop saving the token in localStorage. Instead, store it in a secure cookie with these flags: HttpOnly, Secure, and SameSite=Strict.

Bug 4: Stored HTML / Script Payloads (Low-Medium)

What is the bug?
You can create a Major Incident with JavaScript or HTML in the title and description. The system saves the dangerous code.

You can see:

- Title containing: XSS Test <img src=x onerror=alert(document.domain)>

- An incident created by the Agent ("Agent created MI")

Currently the page shows the code as plain text (React escapes it), so it does not run. But the dangerous data is still stored in the database. If any future page renders it unsafely, it will become a real XSS.

How to fix it (in words):

- Clean or escape all user input before saving it.
- Make sure every place that shows the title or description always treats it as text, never as HTML.
- Add a strong Content Security Policy (CSP).

Bug 5: Public Health Endpoints (Low-Medium)

What is the bug?
Anyone (even without logging in) can open:

- /api/health
- /api/health/db

These pages reveal that the system is running in production and show database status.

How to fix it (in words):
Either remove these endpoints from public access or return only a simple "ok" message. Do not show environment or database details to the public.

Additional Notes

- **Service User** and **Approver** roles are correctly restricted on most sensitive endpoints (they get 403 Forbidden).
- Password reset is safe (it does not reveal whether a username exists).
- Support Team User password that works is Supp0rt!2xL8^pD5 (without the trailing =).

Priority Order to Fix

1. Restrict user list and Organisations page to Admin only (Bug 1)
2. Add rate limiting and account lockout on login (Bug 2)
3. Move JWT token out of localStorage (Bug 3)
4. Sanitize user input in Major Incidents and improve CSP (Bug 4)
5. Lock down health endpoints (Bug 5)

New Bug 1: Mass Assignment / Business Logic Flaw on Major Incidents (High)

What is the bug?
When creating or updating a Major Incident, the API accepts and applies extra fields that should be controlled only by the system:

- You can set status to "Closed" or "Resolved" at creation time
- You can set any commander value (even "hacker")
- You can inject HTML/script into the title

How it was done:

JSON

POST /api/major-incidents

{

"title": "MassAssign Test",

"status": "Closed", ← ***should not be allowed on create***

"commander": "admin-001", ← ***should be system-controlled***

...

}

PATCH /api/major-incidents/{***id***}

{

"status": "Resolved",

"title": "Hijacked Title <script>alert(1)</script>",

"commander": "hacker"

}

Impact
An attacker (or even a normal agent) can force an incident into a final state, change ownership, and store dangerous content. This breaks the intended workflow and audit trail.

How to fix it (in words):

- On create, ignore or reject any client-supplied status, commander, resolvedAt, etc. Always start with status: "Started" and commander: "Unassigned".
- On update, allow only specific fields (e.g. description, updates) and enforce a proper state machine for status changes.
- Sanitize the title and description fields.

New Bug 2: No Server-Side Logout / Token Invalidation (Medium)

What is the bug?
There is **no** /api/auth/logout endpoint.
When a user clicks logout, the frontend only clears localStorage. The JWT remains valid on the server until it expires (several days).

Impact

- Stolen tokens cannot be revoked.
- After logout on one device, the same token still works from another browser/device.
- No way to force-logout a compromised account.

How to fix it (in words):

- Create a proper logout endpoint that blacklists the current token (or the whole user's tokens).
- Use short-lived access tokens + refresh tokens that can be revoked.
- On logout, invalidate the token server-side.

New Bug 3: Token Refresh Does Not Invalidate Old Token (Medium)

What is the bug?
Calling /api/auth/refresh gives a new token, but the **old token continues to work**. Multiple valid tokens can exist for the same user at the same time.

Impact
Increases the window of opportunity if a token is stolen. Makes proper session management harder.

How to fix it (in words):
When a new token is issued, invalidate the previous one (or use a token family / rotation scheme).

New Bug 4: Missing Clickjacking Protection on Frontend (Low-Medium)

What is the bug?
The frontend pages do not send X-Frame-Options or a strong Content-Security-Policy with frame-ancestors.

(The backend API has good headers, but the main application pages do not.)

Impact
The application can be embedded in an attacker's iframe (clickjacking), tricking users into clicking hidden buttons.

How to fix it (in words):
Add these headers on all frontend responses:

- X-Frame-Options: DENY (or SAMEORIGIN)
- Content-Security-Policy: frame-ancestors 'none'; (or 'self')

Test	Result
HTTP Method Tampering on /api/users	Only GET works (good)
Path Traversal (/api/users/../../../etc/passwd)	Blocked
NoSQL Injection on login	Rejected cleanly
Source maps / .env / .git	Not exposed (good)
Verbose error messages	Mostly safe, but ticket-not-found reveals ID existence

Summary of New Findings

Severity	Bug
High	Mass Assignment on Major Incidents (status, commander, title)
Medium	No server-side logout / token revocation
Medium	Refresh does not invalidate previous token
Low-Medium	Missing clickjacking protection on frontend

New Findings from This Round

1. API Accepts Requests from Arbitrary Origins (CSRF-related) (Low-Medium)

What was tested
Sent a state-changing request (POST /api/major-incidents) with header:

text

Origin: https://evil-attacker.com

Result
The API accepted the request and created the Major Incident successfully.

Why it matters
Because the application uses Bearer tokens in localStorage (not cookies), classic browser CSRF is harder. However, if a token is ever leaked or if the app later switches to cookie-based auth, the lack of Origin/Referer validation becomes dangerous.

How to fix

- Validate the Origin (or Referer) header on state-changing requests.

Prefer cookie-based auth with proper CSRF tokens if moving away from localStorage.

2. No Classic Open Redirect Found (Positive)

Tested multiple redirect payloads:

- ?redirect=https://evil.com
- ?redirect=//evil.com
- ?redirect=/\evil.com

The login page loads but does **not** automatically redirect to the external site after authentication. No server-side open redirect was confirmed.

3. Password Reset is Safe (Positive)

- Generic success message for both existing and non-existing users.
- Reset endpoint requires authentication (cannot be abused with a guessed token easily).
- No user enumeration via the reset flow.

4. Client-side JavaScript Analysis (Positive + Notes)

Check	Result
Hardcoded API keys / secrets	None found
innerHTML / dangerouslySetInnerHTML / eval	Not present in the main bundle
Backend URL hard-coded	Yes (https://servigen-backend.onrender.com) – expected
Source maps	Not publicly accessible

Note: Token is still stored in localStorage (already reported earlier).

5. Approvals Workflow Page

The **My Approvals** page is functional and shows pending items (e.g. Knowledge Article Draft).

“Accept All” / “Reject All” buttons are present. No immediate privilege escalation was found on this page during the test, but the bulk actions should be protected by proper server-side checks.

6. Remaining IDOR / Access Control Checks

Objects	Service User	Service Agent	Notes
Change Request CR-001	403 Forbidden	200 OK	Agent can read
Projects	403	200	Consistent with earlier findings
Lists / List Builder	403	200	Agent can see custom lists
Services Catalogue	Accessible to Admin	-	Limited data

7. Information Disclosure via Errors

- Non-existent ticket → "Ticket nonexistent-id-12345 not found" (reveals that the ID format is validated).
- Path traversal attempts are blocked cleanly.
- NoSQL-style injection on login is rejected with a clear type-validation message.

Overall error handling is reasonably safe.

Summary of This Testing Round

Finding	**Severity**	**Status**
API accepts requests from arbitrary Origin	Low-Medium	Confirmed
Open Redirect	-	Not found
Password Reset enumeration	-	Safe
Hardcoded secrets in JS	-	Not found
Dangerous JS sinks (innerHTML/eval)	-	Not found
Approvals bulk actions	Info	Needs server-side verification
Additional IDOR on Change Requests / Lists	Consistent with earlier High	Agent has broad access

New Bug 1: Race Condition on Major Incident Status Updates (Medium-High)

What is the bug?
Multiple concurrent PATCH requests can update the same Major Incident status at the same time. There is no locking or optimistic concurrency control. The last request wins, and intermediate states can be lost.

Test performed:
Fired 5 simultaneous PATCH requests with different statuses (Resolved, Closed, Started, Escalated, Mitigated). All returned success.

Impact

- Lost updates in multi-user environments
- Inconsistent audit trail
- Possible workflow corruption

How to fix it (in words):

- Add optimistic locking (version/ETag field) or pessimistic locking.
- Reject updates if the current status does not match the expected previous status.

Use a proper state machine that only allows valid transitions.

New Bug 2: Change Request Status Can Be Forced (Workflow Bypass) (High)

What is the bug?
An Admin (and possibly other privileged roles) can directly set a Change Request status to "Approved" via API, skipping the normal approval workflow.

Proof:

http

PATCH /api/change-requests/CR-001

{"status": "Approved"}

→ 200 OK, status becomes "Approved"

Original status was "On Hold". After the call it became "Approved" without going through any approval steps.

Impact
Business process bypass. Changes can be marked approved without proper review.

How to fix it (in words):

- Enforce a strict state machine for Change Request statuses.
- Only allow specific transitions (e.g. Pending Approval → Approved) and only by users with the Approver role.

Log every status change with the actor and previous state.

New Bug 3: Duplicate Query Parameters Cause Internal Server Error (500) (Medium)

What is the bug?
Sending duplicate query parameters to /api/tickets triggers a **500 Internal Server Error**.

Proof:

http

GET /api/tickets?status=Open&status=Closed

→ HTTP 500 {"success":false,"message":"Internal server error"}

Also happens with status[]=Open&status[]=Closed.

Impact

- Unhandled exception (possible information disclosure in server logs)
- Potential denial-of-service if an attacker floods the endpoint with malformed queries
- Indicates missing input validation / array handling

How to fix it (in words):

- Properly parse and validate query parameters.
- If multiple values are sent for a single filter, either reject them with 400 or take the first/last value safely.
- Never let unhandled exceptions reach the client as 500.

New Bug 4: Extremely Large Payloads Accepted (Low-Medium)

What is the bug?
A Major Incident title and description of **50,000 characters** each was accepted and stored successfully (HTTP 201).

Impact
Possible storage abuse or performance issues if many large records are created.

How to fix it (in words):

- Enforce reasonable maximum length limits on all text fields (e.g. title ≤ 200–500 chars, description ≤ 5,000–10,000 chars).
- Return a clear 400 error when limits are exceeded.

Other Observations from This Round

Test	Result
SSTI payloads ({{7*7}}, ${7*7}, etc.)	Stored as plain text (not executed) – Good
SQL injection style payloads	Rejected or caused parse errors – mostly safe
Wrong data types (array instead of string, null, etc.)	Correctly rejected with validation message – Good
Agent forcing invalid Change Request status	Rejected with allowed-values list – Good validation
Rate limiting header observed	300-in-1min on some endpoints (login still had none earlier)

Summary of Fourth Round Findings

Severity	Bug
High	Change Request status can be forced to Approved (workflow bypass)
Medium-High	Race condition on concurrent Major Incident status updates
Medium	Duplicate query parameters → 500 Internal Server Error
Low-Medium	No length limits on large text payloads